

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VII

Subject: Data Science Lab

Experiment – 9

1. **Aim:** To Evaluate classification algorithms.
2. **Objectives:** Students should be define and apply metrics to measure the performance of various learning algorithms
3. **Prerequisite:** Python basics
4. **Pre-Experiment Exercise:**
Theory:
Compare
 - i. **Popular Classification Algorithms:**
 - ii. **Logistic Regression**
 - iii. **Naive Bayes**
 - iv. **K-Nearest Neighbors**
 - v. **Decision Tree**
 - vi. **Support Vector Machines**
5. **Laboratory Exercise**
Procedure:
 - i. Compare above listed classifier algorithms and apply metrics to measure the performances of each.
6. **Post-Experiments Exercise:**
A. Extended Theory:
 - i. Explain various performance metrics.
7. **Conclusion:**
 - i. Write what was performed in the program (s) .
 - ii. What is the significance of program and what Objective is achieved?
8. **References:**
 - a. https://scikit-learn.org/stable/modules/cross_validation.html

5. Laboratory Exercise:

Code:

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split,
cross_val_predict, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report,
    roc_curve, auc
)

# Classifiers
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

SEED = 42
RANDOM_STATE = SEED

def load_data():
    data = load_breast_cancer()
    X = data.data
    y = data.target
    feature_names = data.feature_names
    target_names = data.target_names
    return X, y, feature_names, target_names

def make_classifiers():
    # Use pipelines where scaling is required (LR, KNN, SVM)
    classifiers = {
        "LogisticRegression": Pipeline([
            ("scaler", StandardScaler()),
            ("clf",
             LogisticRegression(random_state=RANDOM_STATE,
                                max_iter=500))
        ]),
        "NaiveBayes": Pipeline([
            # GaussianNB works fine without scaling, but pip
            included for consistency
            ("scaler", StandardScaler()),
            ("clf", GaussianNB())
        ]),
        "KNN": Pipeline([
            ("scaler", StandardScaler()),
            ("clf", KNeighborsClassifier(n_neighbors=5))
        ]),
        "DecisionTree": Pipeline([
            ("clf",
             DecisionTreeClassifier(random_state=RANDOM_STATE))
        ]),
        "SVM": Pipeline([
            ("scaler", StandardScaler()),
            # probability=True for ROC AUC and probability
            estimates (slower)
            ("clf", SVC(kernel='rbf', probability=True,
                        random_state=RANDOM_STATE))
        ])
    }
    return classifiers

def evaluate_classifier(clf, X_train, X_test, y_train, y_test):
    stats = {}
    t0 = time.time()
    clf.fit(X_train, y_train)
    train_time = time.time() - t0

    t1 = time.time()
    y_pred = clf.predict(X_test)
    predict_time = time.time() - t1

    # Some classifiers return probabilities; handle both prob and
    decision_function
    y_score = None
    try:
        # prefer predict_proba if available
        y_score = clf.predict_proba(X_test)[:, 1]
    except Exception:
        try:
            y_score = clf.decision_function(X_test)
        except Exception:
            # fallback: use predicted labels (will make ROC AUC
            not meaningful but keep code safe)
            y_score = y_pred

    stats['accuracy'] = accuracy_score(y_test, y_pred)
    stats['precision'] = precision_score(y_test, y_pred)
    stats['recall'] = recall_score(y_test, y_pred)
    stats['f1'] = f1_score(y_test, y_pred)

    # For ROC AUC: if y_score is constant, roc_auc_score will
    raise error; guard it.
    try:
        stats['roc_auc'] = roc_auc_score(y_test, y_score)
    except Exception:
        stats['roc_auc'] = np.nan

    stats['train_time_sec'] = train_time
    stats['predict_time_sec'] = predict_time
    stats['y_pred'] = y_pred
    stats['y_score'] = y_score
    stats['confusion_matrix'] = confusion_matrix(y_test, y_pred)
    stats['classification_report'] = classification_report(y_test,
                                                            y_pred, digits=4)
    return stats

def main():
    X, y, feature_names, target_names = load_data()
    print(f'Dataset: Breast Cancer Wisconsin
          (n_samples={X.shape[0]}, n_features={X.shape[1]})')
```

```

print("Target names:", target_names)

# split (stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y,
    random_state=RANDOM_STATE
)

classifiers = make_classifiers()

results = {}
for name, clf in classifiers.items():
    print(f"\nTraining & evaluating: {name}")
    stats = evaluate_classifier(clf, X_train, X_test, y_train, y_test)
    results[name] = stats
    print(f" Accuracy: {stats['accuracy']:.4f}, Precision: {stats['precision']:.4f}, Recall: {stats['recall']:.4f}, F1: {stats['f1']:.4f}, ROC AUC: {stats['roc_auc']:.4f}")
    print(" Confusion matrix:\n", stats['confusion_matrix'])
    print("\nClassification report:\n", stats['classification_report'])

# Summary table
summary_rows = []
for name, stats in results.items():
    summary_rows.append({
        'Classifier': name,
        'Accuracy': stats['accuracy'],
        'Precision': stats['precision'],
        'Recall': stats['recall'],
        'F1': stats['f1'],
        'ROC_AUC': stats['roc_auc'],
        'Train_time_s': stats['train_time_sec'],
        'Predict_time_s': stats['predict_time_sec']
    })
summary_df =
pd.DataFrame(summary_rows).sort_values(by='F1',
ascending=False).reset_index(drop=True)

print("\n\n=== Summary (sorted by F1) ===")
print(summary_df.to_string(index=False,
float_format="{:0.4f}".format))

# Plot ROC curves on same plot
plt.figure(figsize=(8, 6))
for name, stats in results.items():
    y_test = y_test_local = y_test # same in this script
    y_score = stats['y_score']
    # ensure y_score is 1d numeric
    try:
        fpr, tpr, _ = roc_curve(y_test_local, y_score)
        roc_auc = auc(fpr, tpr)
        plt.plot(fpr, tpr, lw=2, label=f"{name} (AUC = {roc_auc:.3f})")

```

```

except Exception:
    # skip if ROC can't be computed
    print(f"Could not compute ROC for {name}")

plt.plot([0, 1], [0, 1], linestyle='--', lw=1, label='Chance')
plt.xlim([-0.02, 1.0])
plt.ylim([0.0, 1.02])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves - Classifier Comparison')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()

# Optional: cross-validated metrics (StratifiedKfold)
print("\nPerforming 5-fold cross-validated accuracy comparison (StratifiedKfold)...")
skf = StratifiedKfold(n_splits=5, shuffle=True,
random_state=RANDOM_STATE)
cv_summary = []
for name, clf in classifiers.items():
    # cross_val_predict for predicted probabilities (for ROC) or labels
    print(f" Cross-validating {name} ...")
    start = time.time()
    # We'll compute cross-validated accuracy using cross_val_predict labels
    y_pred_cv = cross_val_predict(clf, X, y, cv=skf,
method='predict', n_jobs=None)
    elapsed = time.time() - start
    acc_cv = accuracy_score(y, y_pred_cv)
    prec_cv = precision_score(y, y_pred_cv)
    rec_cv = recall_score(y, y_pred_cv)
    f1_cv = f1_score(y, y_pred_cv)
    cv_summary.append({
        'Classifier': name,
        'CV_Accuracy': acc_cv,
        'CV_Precision': prec_cv,
        'CV_Recall': rec_cv,
        'CV_F1': f1_cv,
        'CV_time_s': elapsed
    })
    print(f" CV Accuracy: {acc_cv:.4f}, CV F1: {f1_cv:.4f} (time {elapsed:.2f}s)")

cv_df =
pd.DataFrame(cv_summary).sort_values(by='CV_F1',
ascending=False)
print("\nCross-validated summary:")
print(cv_df.to_string(index=False,
float_format="{:0.4f}".format))

if __name__ == "__main__":
    main()

```

Output:

```
Dataset: Breast Cancer Wisconsin (n_samples=569, n_features=30)
Target names: ['malignant' 'benign']

Training & evaluating: LogisticRegression
Accuracy: 0.9860, Precision: 0.9889, Recall: 0.9889, F1: 0.9889, ROC AUC: 0.9977
Confusion matrix:
[[52  1]
 [ 1 89]]

Classification report:
      precision    recall  f1-score   support

     0       0.9811      0.9811      0.9811         53
     1       0.9889      0.9889      0.9889         90

   accuracy          0.9850      0.9850      0.9860        143
  macro avg          0.9850      0.9850      0.9850        143
weighted avg          0.9860      0.9860      0.9860        143
```

```
Training & evaluating: NaiveBayes
Accuracy: 0.9371, Precision: 0.9355, Recall: 0.9667, F1: 0.9508, ROC AUC: 0.9878
Confusion matrix:
[[47  6]
 [ 3 87]]

Classification report:
      precision    recall  f1-score   support

     0       0.9400      0.8868      0.9126         53
     1       0.9355      0.9667      0.9508         90

   accuracy          0.9371      0.9371      0.9371        143
  macro avg          0.9377      0.9267      0.9317        143
weighted avg          0.9372      0.9371      0.9367        143
```

```
Training & evaluating: KNN
Accuracy: 0.9790, Precision: 0.9677, Recall: 1.0000, F1: 0.9836, ROC AUC: 0.9845
Confusion matrix:
[[50  3]
 [ 0 90]]

Classification report:
      precision    recall  f1-score   support

     0       1.0000      0.9434      0.9709         53
     1       0.9677      1.0000      0.9836         90

   accuracy          0.9839      0.9717      0.9772        143
  macro avg          0.9839      0.9717      0.9772        143
weighted avg          0.9797      0.9790      0.9789        143
```

```
Training & evaluating: DecisionTree
Accuracy: 0.9231, Precision: 0.9540, Recall: 0.9222, F1: 0.9379, ROC AUC: 0.9234
Confusion matrix:
[[49  4]
 [ 7 83]]

Classification report:
      precision    recall  f1-score   support

     0       0.8750      0.9245      0.8991         53
     1       0.9540      0.9222      0.9379         90

   accuracy          0.9145      0.9234      0.9185        143
  macro avg          0.9145      0.9234      0.9185        143
weighted avg          0.9247      0.9231      0.9235        143
```

```
Training & evaluating: SVM
Accuracy: 0.9790, Precision: 0.9888, Recall: 0.9778, F1: 0.9832, ROC AUC: 0.9969
Confusion matrix:
[[52  1]
 [ 2 88]]

Classification report:
      precision    recall  f1-score   support

     0       0.9630      0.9811      0.9720         53
     1       0.9888      0.9778      0.9832         90

   accuracy          0.9759      0.9795      0.9776        143
  macro avg          0.9759      0.9795      0.9776        143
weighted avg          0.9792      0.9790      0.9791        143
```

=== Summary (sorted by F1) ===							
Classifier	Accuracy	Precision	Recall	F1	ROC_AUC	Train_time_s	Predict_time_s
LogisticRegression	0.9860	0.9889	0.9889	0.9889	0.9977	0.1021	0.0010
KNN	0.9790	0.9677	1.0000	0.9836	0.9845	0.0025	0.1000
SVM	0.9790	0.9888	0.9778	0.9832	0.9969	0.0282	0.0076
NaiveBayes	0.9371	0.9355	0.9667	0.9508	0.9878	0.0075	0.0011
DecisionTree	0.9231	0.9540	0.9222	0.9379	0.9234	0.0395	0.0009

